Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»
Факультет компьютерных систем и сетей
Кафедра информатики

Дисциплина «Операционные среды и системное программирование»

ОТЧЕТ

к лабораторной работе №4

на тему:

«Обедающие философы: справедливое распределение ресурсов»

БГУИР 6-05-0612-02 28

Выполнил студент группы 453503
ВАСИЛЕНКО Дмитрий Васильевич
_____

(дата, подпись студента)

Проверил ассистент каф. информатики
ШАБАНОВИЧ Роман Александрович
_____

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 НОМЕР И НАЗВАНИЕ ВАРИАНТА ... 3
2 ЦЕЛЬ РАБОТЫ ... 4
3 АРХИТЕКТУРА ПРОГРАММНОГО КОМПЛЕКСА ... 5
3.1 Структура проекта ... 5
3.2 Потоки и общие ресурсы ... 5
3.3 Жизненный цикл заявки ... 6
3.4 Схема взаимодействия ... 6
3.5 Индивидуальные параметры и конфигурации ... 9
4 ИСПОЛЬЗУЕМЫЕ МЕХАНИЗМЫ СИНХРОНИЗАЦИИ ... 10
4.1 Соответствие Windows API и Linux/POSIX ... 10
4.2 Критическая секция ... 10
4.3 Event на каждую заявку ... 10
4.4 Центральная очередь и режимы FIFO/SCAN ... 11
4.5 Тайм-ауты, завершение и watchdog ... 11
4.6 Перечень использованных системных вызовов ... 11
4.7 Обработка ошибок ... 12
5 ЖУРНАЛ И СТАТИСТИКА ... 13
5.1 Формат журнала ... 13
5.2 Итоговая сводка и CSV ... 13
6 ПРОВЕРКА КОРРЕКТНОСТИ ... 14
6.1 Проверяемые инварианты ... 14
6.2 Аргументы корректности ... 14
6.3 Автоматические проверки и аварийные сценарии ... 15
7 ФРАГМЕНТЫ КОДА ... 16
7.1 Заявка и событие ... 16
7.2 Выдача вилок в режимах FIFO и SCAN ... 16
7.3 Ожидание с тайм-аутом ... 17
7.4 Основной цикл философа ... 18
7.5 Финальная проверка ... 19
8 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ ... 20
8.1 Методика ... 20
8.2 Конфигурации ... 20
8.3 Результаты ... 20
8.4 Графики ... 21
8.5 Анализ результатов ... 23
9 ЗАКЛЮЧЕНИЕ ... 25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ... 26
ПРИЛОЖЕНИЕ А.
ИСПОЛЬЗОВАНИЕ МОДЕЛЕЙ
ИСКУССТВЕННОГО ИНТЕЛЛЕКТА ... 27
ПРИЛОЖЕНИЕ Б.
ЖУРНАЛ ДЕМОНСТРАЦИОННОГО ЗАПУСКА ... 28

1 НОМЕР И НАЗВАНИЕ ВАРИАНТА

Вариант 6. Модель «Обедающие философы»: справедливое распределение ресурсов.

Формулировка задания:

- за круглым столом сидят N философов; между соседними философами лежит одна вилка, всего N вилок;
- для приёма пищи философу нужны обе соседние вилки;
- философ циклически переходит между состояниями обдумывания, ожидания и приёма пищи;
- запросы на вилки должны обслуживаться через центральную очередь;
- необходимо реализовать справедливый режим `fifo` и режим с возможным обходом заблокированной заявки `scan`;
- требуется провести не менее трёх запусков для каждой конфигурации, измерить пропускную способность и распределение времени ожидания;
- по результатам работы построить не менее двух графиков и подготовить отчёт.

Дополнительно моделируются тайм-аут ожидания, максимальная длительность эксперимента, аварийное завершение и проверка инвариантов. Работа выполнена в Ubuntu Linux, поэтому вместо Windows API используются их функциональные аналоги POSIX/C++: критическая секция заменена мьютексом, auto-reset event — семафором, `WaitForSingleObject` — `sem_timedwait()`, `GetCurrentThreadId` — `gettid()`.

Индивидуальные параметры. Номер студенческого билета $S = 28$. По формулам задания:

- $\mathrm{SEED} = 1000 + 97S = 1000 + 97 \cdot 28 = 3716$;
- $\mathrm{OPERATIONS} = 100 + 5S = 100 + 5 \cdot 28 = 240$ циклов на философа;
- стандартная конфигурация — 5 философов по 100 циклов;
- индивидуальная конфигурация — 5 философов по 240 циклов.

2 ЦЕЛЬ РАБОТЫ

Целью работы является изучение синхронизации потоков, критических секций, событий и централизованных очередей заявок, а также исследование справедливости распределения ограниченных ресурсов на примере задачи «Обедающие философы».

В ходе работы необходимо:

- реализовать модель с N потоками-философами и N вилками;
- обеспечить атомарный захват двух вилок и исключить взаимоблокировку;
- реализовать центральную очередь заявок и два режима обслуживания: `fifo` и `scan`;
- реализовать ожидание на event с тайм-аутом и корректное освобождение ресурсов при аварийном завершении;
- проверить инварианты и отсутствие гонок при многократных запусках;
- измерить пропускную способность и распределение времени ожидания;
- сравнить FIFO и SCAN, а также оценить влияние числа философов;
- подготовить воспроизводимые эксперименты, графики и отчёт.

3 АРХИТЕКТУРА ПРОГРАММНОГО КОМПЛЕКСА

3.1 Структура проекта

Программа собирается в один исполняемый файл `lab04_sync` и не требует внешних библиотек, кроме стандартной библиотеки C++ и POSIX. Структура проекта приведена в таблице 1.

Таблица 1 – Структура проекта

Путь	**Назначение**
`lab04-sync/src/main.cpp`	реализация модели, синхронизации, CLI, статистики и проверки инвариантов
`lab04-sync/CMakeLists.txt`	сборка, подключение CTest и Python-проверок
`lab04-sync/tests/run_checks.py`	автоматические проверки корректности, тайм-аутов и аварийных сценариев
`lab04-sync/scripts/run_experiments.py`	запуск шести конфигураций эксперимента по три раза
`lab04-sync/scripts/plot_results.py`	построение графиков по `results.csv` и `waits.csv`
`lab04-sync/data/`	сырые данные, сводные CSV, журналы и графики
`lab04-sync/diagrams/`	исходные и отрисованные диаграммы
`lab04-sync/docs/architecture.md`	подробное описание архитектуры
`lab04-sync/report/`	исходный текст и PDF отчёта

3.2 Потоки и общие ресурсы

Комплекс работает в одном процессе. Основные участники и разделяемые ресурсы перечислены в таблице 2.

Таблица 2 – Потоки и общие ресурсы

Участник или ресурс	**Назначение**
Главный поток	разбор аргументов, запуск философов, `join()`, итоговая сводка
Потоки-философы `P0 … P(N-1)`	выполняют циклы обдумывания, ожидания и еды; каждый имеет уникальный TID
Watchdog	проверяет максимальную длительность эксперимента и устанавливает флаг остановки
`fork_owner[N]`	владелец каждой вилки; значение `-1` означает, что вилка свободна
Центральная очередь	`std::deque<Request*>`; хранит ожидающие заявки в порядке поступления
Состояния философов	`THINKING`, `WAITING`, `EATING`, `FINISHED`, `ABORTED`
Статистика	число выдач, тайм-аутов, обходов, времена ожидания, максимальная длина очереди
Журнал событий	потокобезопасный текстовый журнал с отдельным мьютексом

Все общие данные, кроме журнала, защищены одним мьютексом `cs`, который играет роль `CRITICAL_SECTION`. Журнал имеет собственный мьютекс,

чтобы запись в файл не увеличивала время удержания основной критической секции.

3.3 Жизненный цикл заявки

Философ выполняет заданное число циклов. Один цикл состоит из следующих шагов.

1. Философ находится в состоянии `THINKING` и имитирует обдумывание.
2. По окончании обдумывания создаётся объект `Request`, содержащий сразу обе соседние вилки, и инициализируется персональный семафор-событие.
3. Под мьютексом `cs` философ переходит в `WAITING`, помещает заявку в конец центральной очереди и вызывает `dispatch_locked()`.
4. Если заявку можно удовлетворить, `dispatch_locked()` назначает владельцев вилок, переводит заявку в `GRANTED`, удаляет её из очереди и выполняет `sem_post()`. Философ переходит в `EATING`.
5. Если заявку нельзя удовлетворить сразу, философ ожидает на своём event с абсолютным тайм-аутом. Сигнал, отправленный до начала ожидания, не теряется, потому что семафор сохраняет значение 1.
6. После еды философ под мьютексом освобождает обе вилки, увеличивает счётчик выполненных циклов и снова вызывает `dispatch_locked()`, чтобы передать вилки ожидающим.
7. После выполнения всех циклов философ переходит в `FINISHED`.

Заявка всегда содержит обе вилки. Поэтому философ либо получает оба ресурса, либо не получает ни одного; удержание одной вилки в ожидании второй невозможно. Это исключает классическую взаимоблокировку.

3.4 Схема взаимодействия

На рисунке 1 показаны основные компоненты программы и связи между ними. Главный поток создаёт философов и ожидает их завершения. Философы обращаются к центральной очереди и владельцам вилок только через критическую секцию. Для каждой заявки используется отдельный event, на котором философ ожидает решения.

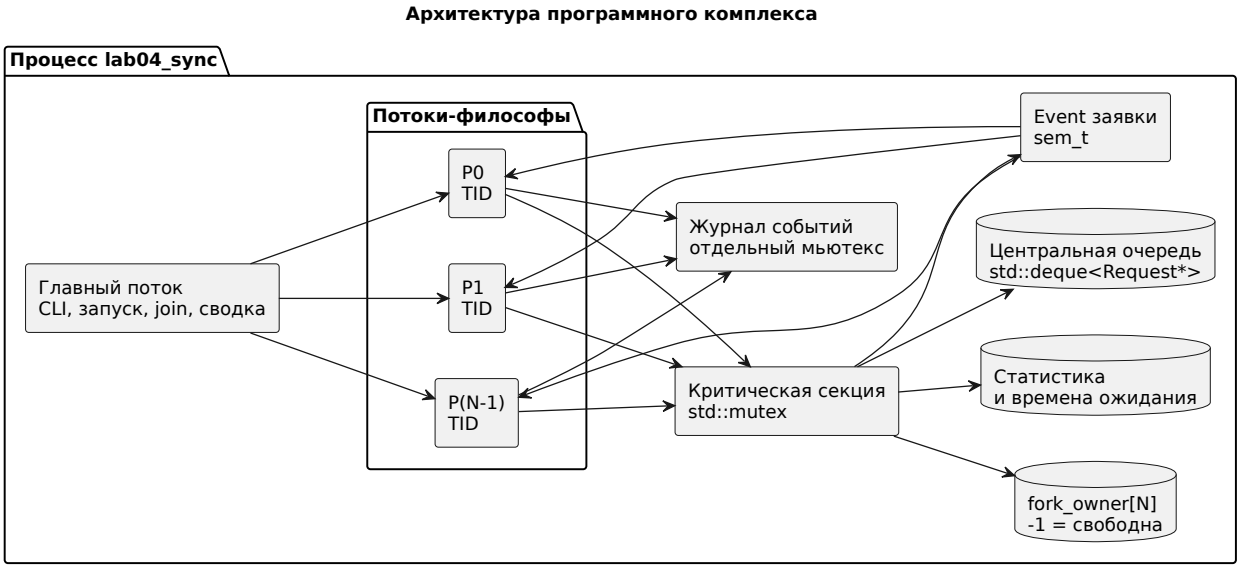


Рисунок 1 – Архитектура программного комплекса

На рисунке 2 приведена диаграмма состояний философа. Переход из `WAITING` в `THINKING` по тайм-ауту означает отмену заявки и повторную попытку позднее.

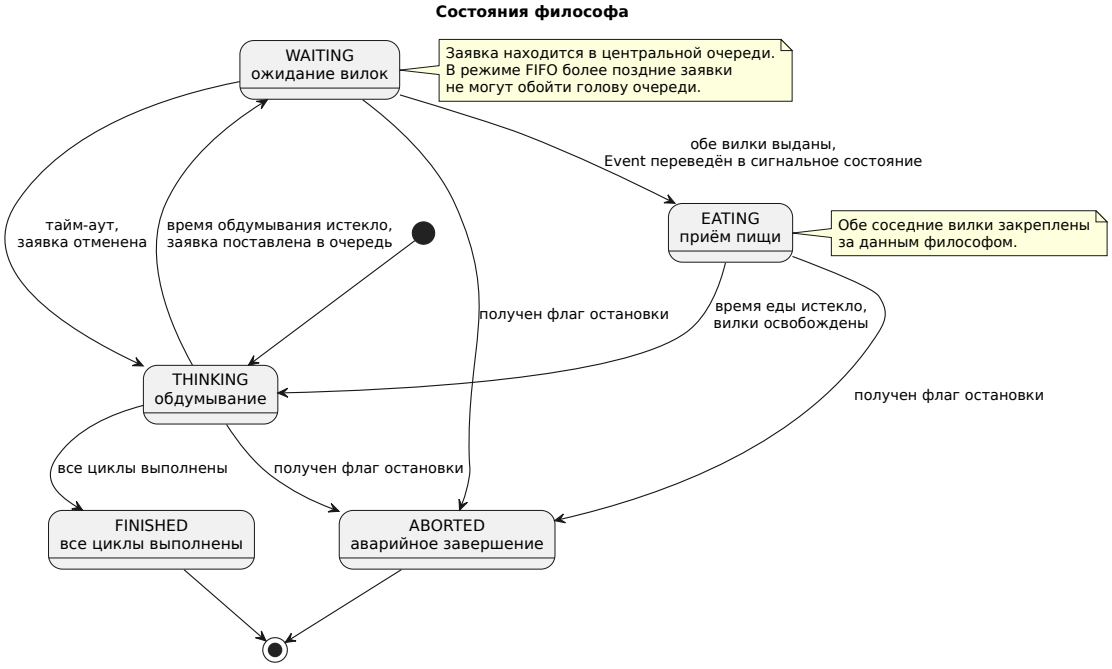


Рисунок 2 – Состояния философа

Последовательность обработки заявки в режиме FIFO показана на рисунке 3. Философ P1 освобождает вилки, после чего `dispatch_locked()` выдаёт вилки первой заявке очереди — запросу философа P0.

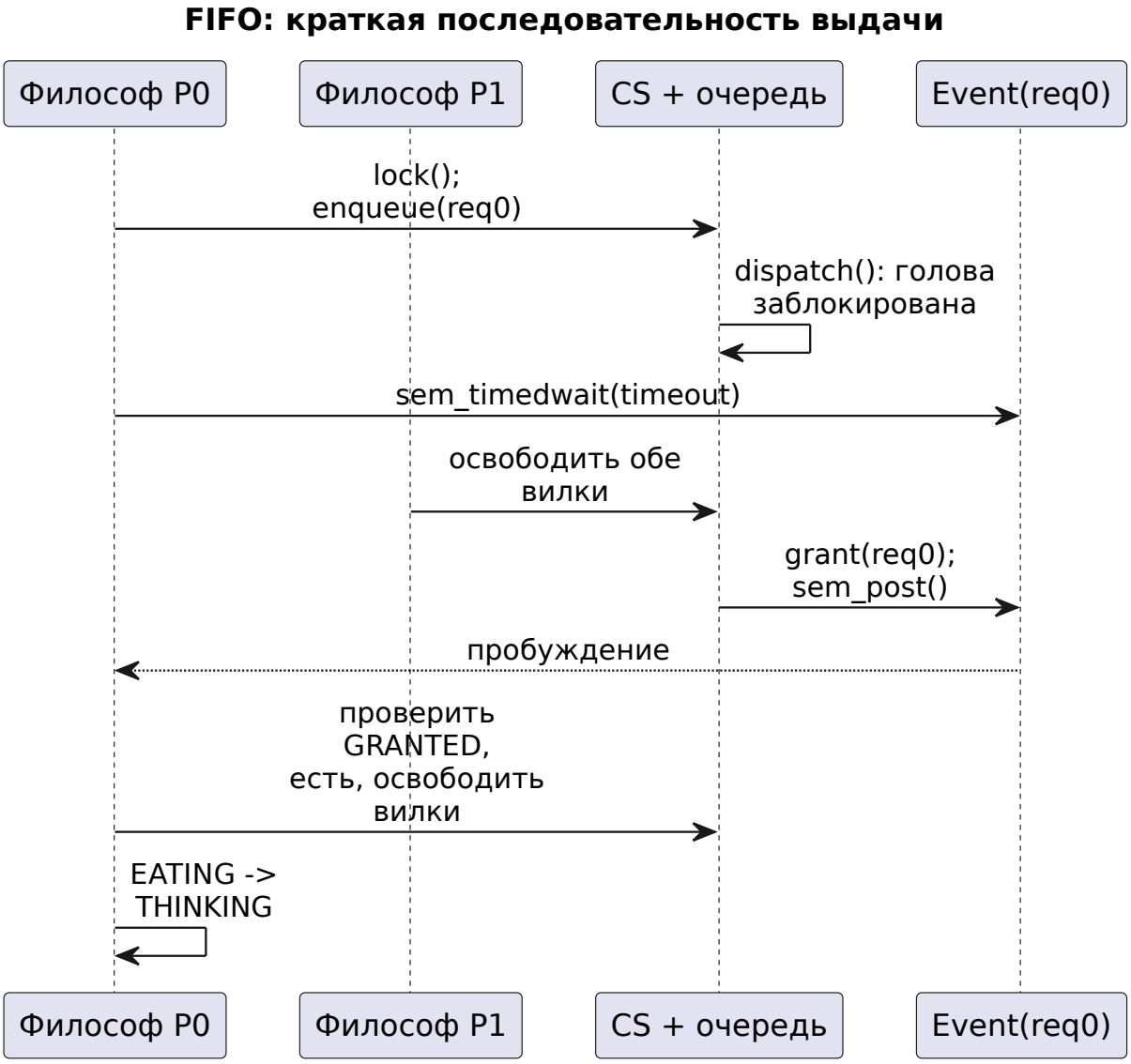


Рисунок 3 – Краткая последовательность FIFO

На рисунке 4 показан режим SCAN. Заявки P0 и P1 не могут быть удовлетворены, поэтому при постановке заявки P2 функция `dispatch_locked()` просматривает очередь с начала, пропускает заблокированные заявки и выдаёт вилки P2. Счётчик `overtakes` увеличивается.

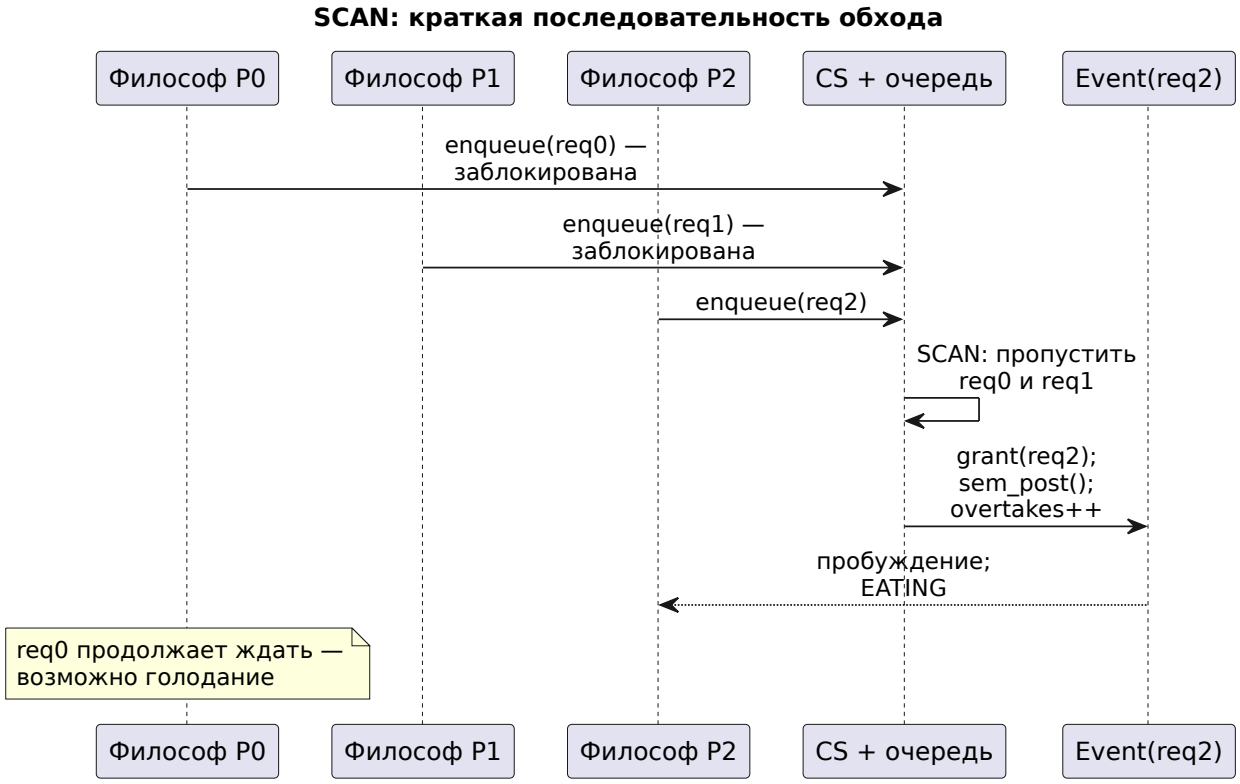


Рисунок 4 – Краткая последовательность SCAN

3.5 Индивидуальные параметры и конфигурации

Значения индивидуальных параметров и основные ограничения приведены в таблице 3. Все эксперименты используют одно и то же зерно генератора, поэтому распределение длительностей обдумывания и еды воспроизводимо.

Таблица 3 – Индивидуальные параметры

Параметр	**Значение**	**Пояснение**
`STUDENT_ID`	`28`	номер студенческого билета
`SEED`	`3716`	$1000 + 97 \cdot 28$
`OPERATIONS`	`240`	$100 + 5 \cdot 28$ циклов на философа
Стандартная конфигурация	5 × 100	5 философов, 100 циклов
Индивидуальная конфигурация	5 × 240	5 философов, 240 циклов
Тайм-аут заявки	30 000 мс	по умолчанию
Максимальная длительность	180 000 мс	по умолчанию

4 ИСПОЛЬЗУЕМЫЕ МЕХАНИЗМЫ СИНХРОНИЗАЦИИ

4.1 Соответствие Windows API и Linux/POSIX

Работа выполняется в Linux, поэтому использованы функциональные аналоги механизмов Windows API. Соответствие приведено в таблице 4.

Таблица 4 – Соответствие механизмов Windows API и Linux/POSIX

Windows API	**Linux/POSIX или C++**	**Назначение**
`CRITICAL_SECTION`	`std::mutex` (`pthread_mutex_t`)	защита общих данных
`CreateEvent` (auto-reset)	`sem_init()`	создание события заявки
`SetEvent`	`sem_post()`	перевод события в сигнальное состояние
`WaitForSingleObject`	`sem_timedwait()`	ожидание события с тайм-аутом
`CloseHandle`	`sem_destroy()`	удаление события
`GetCurrentThreadId`	`syscall(SYS_gettid)`	идентификатор потока
`Sleep`	`std::this_thread::sleep_for()`	имитация обдумывания и еды
`InterlockedIncrement`	`std::atomic`	флаги остановки и аварии

`Sleep`/`sleep_for` используется только для имитации времени обдумывания и еды. Для согласования доступа к вилкам сон не применяется.

4.2 Критическая секция

Один мьютекс `cs` защищает:

- очередь заявок;
- массив владельцев вилок `fork_owner`;
- состояния философов;
- счётчики статистики и времена ожидания;
- проверку и изменение статусов заявок.

Проверка «обе вилки свободны» и назначение владельцев выполняются в одной критической секции. Поэтому между проверкой и захватом не может вклиниться другой поток. Все изменения `fork_owner`, `states`, `queue` и статистики выполняются только под `cs`.

4.3 Event на каждую заявку

Для каждой заявки создаётся отдельный семафор `sem_t` с начальным значением

0. Он играет роль auto-reset event:

- `sem_post()` соответствует `SetEvent()`;
- `sem_timedwait()` соответствует `WaitForSingleObject()` с тайм-аутом;
- после успешного ожидания семафор автоматически обнуляется;
- сигнал, отправленный до вызова `sem_timedwait()`, сохраняется и не теряется.

Отдельный семафор на заявку позволяет будить только нужного философа и естественно реализует тайм-аут. Семафор уничтожается в деструкторе объекта `Request` после завершения попытки.

4.4 Центральная очередь и режимы FIFO/SCAN

Очередь хранит указатели на активные заявки. Она защищена тем же мьютексом `cs`. В режиме FIFO функция `dispatch_locked()` рассматривает только голову очереди:

- если обе вилки головы свободны, заявка выдаётся и удаляется из очереди;
- если голова заблокирована, цикл прерывается, и более поздние заявки не обходят её.

В режиме SCAN функция просматривает очередь от начала к концу и выдаёт все заявки, которые можно удовлетворить. Если выданная заявка не была первой, счётчик `overtakes` увеличивается. Такой режим повышает загрузку ресурсов, но допускает голодание: заявка в начале очереди может долго ждать, пока более поздние заявки получают вилки.

4.5 Тайм-ауты, завершение и watchdog

Ожидание выполняется с абсолютным дедлайном. При истечении тайм-аута философ под `cs` проверяет статус заявки:

- если заявка уже `GRANTED`, философ продолжает работу;
- если заявка всё ещё `WAITING`, она удаляется из очереди, переводится в `CANCELLED`, а счётчик тайм-аутов увеличивается.

После отмены вызывается `dispatch_locked()`, потому что следующая заявка могла стать доступной. Проверка статуса и отмена выполняются под тем же мьютексом, что и выдача, поэтому ситуация «выдача одновременно с отменой» разрешается однозначно.

Watchdog проверяет время с шагом 50 мс. При превышении `--max-duration-ms` устанавливаются флаги `stop` и `aborted`; философы завершают циклы, освобождают захваченные вилки и переходят в `ABORTED`. Программа возвращает код 3. Такой запуск не считается корректным экспериментальным результатом.

4.6 Перечень использованных системных вызовов

В таблице 5 перечислены системные вызовы и библиотечные средства, которые обеспечивают модель.

Таблица 5 – Использованные системные вызовы и средства

Средство	**Назначение**	**Аналог Windows**
`pthread_create`, `pthread_join`	создание и ожидание потоков через `std::thread`	`CreateThread`, `WaitForSingleObject`
`pthread_mutex_lock/unlock`	вход и выход из критической секции через `std::mutex`	`EnterCriticalSection`, `LeaveCriticalSection`
`sem_init`	создание auto-reset события	`CreateEvent`
`sem_post`	сигнал о выдаче вилок	`SetEvent`
`sem_timedwait`	ожидание события с тайм-аутом	`WaitForSingleObject`
`sem_destroy`	удаление события	`CloseHandle`
`clock_gettime(CLOCK_REALTIME)`	абсолютный дедлайн для `sem_timedwait`	таймеры Windows
`syscall(SYS_gettid)`	идентификатор потока	`GetCurrentThreadId`
`std::this_thread::sleep_for`	имитация обдумывания и еды	`Sleep`
`std::atomic`	флаги `stop` и `aborted`	`Interlocked*`
`std::chrono::steady_clock`	измерение времени	`QueryPerformanceCounter`

4.7 Обработка ошибок

Программа проверяет:

- корректность аргументов командной строки и диапазонов значений;
- результаты создания потоков;
- инициализацию и уничтожение семафоров;
- открытие журнала и CSV;
- ошибки `clock_gettime()` и `sem_timedwait()`;
- тайм-ауты заявок;
- превышение максимальной длительности;
- нарушения инвариантов и неполное завершение циклов.

При ошибке печатается диагностика, захваченные ресурсы освобождаются, а программа возвращает ненулевой код. Коды возврата: 0 — успех, 1 — ошибка ввода-вывода или потока, 2 — нарушение инварианта или неверные аргументы, 3 — аварийное завершение по watchdog.

5 ЖУРНАЛ И СТАТИСТИКА

5.1 Формат журнала

Журнал содержит относительное время в миллисекундах, номер философа, тип события и, при необходимости, ресурсы. Формат записи:

```
[000001 ms] P1 WAITING RESOURCE=1,2
[000001 ms] P1 ACQUIRED RESOURCE=1,2
[000001 ms] P1 EATING
[000028 ms] P1 RELEASED RESOURCE=1,2
```

Основные типы событий: `START`, `THINKING`, `WAITING`, `ACQUIRED`, `EATING`, `RELEASED`, `TIMEOUT`, `FINISHED`, `ABORTED`. Запись защищена отдельным мьютексом; при измерении производительности подробный журнал отключается, чтобы не искажать время.

5.2 Итоговая сводка и CSV

После завершения программа выводит:

- конфигурацию, время и пропускную способность;
- число завершённых циклов и тайм-аутов;
- среднее и максимальное время ожидания;
- максимальную длину очереди;
- число выдач и обходов;
- число нарушений инвариантов и итоговый статус;
- статистику по каждому философу.

Для дальнейшего анализа сохраняется CSV с колонками `philosopher,wait_ms,result`. Результаты экспериментов агрегируются в `data/results.csv` и `data/waits.csv`.

6 ПРОВЕРКА КОРРЕКТНОСТИ

6.1 Проверяемые инварианты

В программе проверяются следующие инварианты:

1. Одна вилка не может одновременно принадлежать двум философам.
2. Философ в состоянии `EATING` владеет обеими своими вилками.
3. Количество занятых вилок равно удвоенному числу едящих философов.
4. В очереди не больше N заявок и нет дубликатов заявок одного философа.
5. В режиме FIFO отсутствуют обходы (`overtakes == 0`).
6. После завершения очередь пуста, все вилки свободны, все философы находятся в `FINISHED`, все циклы выполнены.

Проверка выполняется при выдаче вилок, при освобождении, при выходе потока и в финальном состоянии. Нарушения увеличивают счётчик `INVARIANT_VIOLATIONS`; корректный запуск должен завершаться с нулевым значением.

6.2 Аргументы корректности

Взаимное исключение. Все проверки и изменения `fork_owner`, очереди и состояний выполняются под одним мьютексом. Поэтому проверка пары вилок и назначение владельцев являются атомарной операцией.

Отсутствие взаимоблокировки. Заявка содержит обе вилки и не может удерживать один ресурс, ожидая второй. Философ либо получает обе вилки, либо ждёт в очереди. Дополнительно тайм-аут ограничивает время ожидания.

Отсутствие потерянного пробуждения. Каждая заявка имеет отдельный семафор. Если `sem_post()` выполнен до `sem_timedwait()`, значение семафора сохраняется, и ожидание завершается немедленно. Статус заявки проверяется под мьютексом, поэтому выдача и отмена не могут произойти одновременно неоднозначно.

Отсутствие утечки вилок. Освобождение выполняется и после нормальной еды, и в аварийной ветке. Финальная проверка требует, чтобы все вилки были свободны, очередь пуста, а все философы завершили циклы.

Справедливость FIFO. В режиме FIFO выдаётся только голова очереди; если голова заблокирована, более поздние заявки не рассматриваются. Поэтому `overtakes` всегда равен нулю.

6.3 Автоматические проверки и аварийные сценарии

Проверки запускаются через CTest и включают:

- корректность FIFO и отсутствие обходов;
- корректность SCAN и наличие обходов при блокировке головы;
- тайм-ауты и повторные попытки;
- превышение максимальной длительности;
- неверные аргументы;
- проверку журнала, CSV и итоговых инвариантов.

Результаты аварийных запусков приведены в таблице 6. Все они завершились предсказуемым кодом и не оставили захваченных вилок.

Таблица 6 – Результаты аварийных сценариев

Сценарий	**Параметры**	**Результат**	**Нарушения**	**Код**
Тайм-аут	5×2, `timeout=1 мс`, еда 200 мс	2007 тайм-аутов, 10/10 циклов, `STATUS=OK`	0	0
Максимальная длительность	5×100, `max-duration=1 мс`	7/500 циклов, `STATUS=ABORTED`	0	3
Неверные аргументы	`--philosophers 1`	сообщение об ошибке	—	2
CTest	все проверки	1/1 passed	0	0

Отдельно проверен сценарий `timeout=1 мс` при длительной еде: программа совершила 2007 повторных попыток, но все 10 циклов завершились, инварианты не нарушены, возвращён код 0. Это подтверждает, что тайм-аут не приводит к потере ресурсов или зависанию.

7 ФРАГМЕНТЫ КОДА

Ниже приведены ключевые фрагменты реализации. Полный исходный текст находится в `lab04-sync/src/main.cpp`.

7.1 Заявка и событие

```
struct Request {
    int philosopherId = 0;
    int leftFork = 0;
    int rightFork = 0;
    RequestStatus status = RequestStatus::Waiting;
    TimePoint enqueuedAt;
    TimePoint grantedAt;
    sem_t event{};
    bool semaphoreReady = false;

    Request(int id, int left, int right)
        : philosopherId(id), leftFork(left), rightFork(right) {
        semaphoreReady = sem_init(&event, 0, 0) == 0;
    }

    ~Request() {
        if (semaphoreReady) {
            sem_destroy(&event);
        }
    }
};
```

7.2 Выдача вилок в режимах FIFO и SCAN

```
void dispatchLocked(SharedState& state, Logger& logger) {
    if (state.stop.load()) {
        return;
    }

    if (state.config.mode == Mode::Fifo) {
        while (!state.queue.empty()) {
            Request* request = state.queue.front();
            if (state.forkOwner[request->leftFork] == -1 &&
                state.forkOwner[request->rightFork] == -1) {
                state.queue.pop_front();
                grantLocked(state, logger, request);
            } else {
                break;
            }
        }
        return;
    }

    auto request = state.queue.begin();
    while (request != state.queue.end()) {
        Request* current = *request;
        if (state.forkOwner[current->leftFork] == -1 &&
            state.forkOwner[current->rightFork] == -1) {
            if (request != state.queue.begin()) {
                ++state.stats[current->philosopherId].overtakes;
```

```
                ++state.totalOvertakes;
            }
            request = state.queue.erase(request);
            grantLocked(state, logger, current);
        } else {
            ++request;
        }
    }
}
```

7.3 Ожидание с тайм-аутом

```
bool waitForGrant(SharedState& state, Request& request, int timeoutMs,
                  Logger& logger) {
    const auto timeout = Milliseconds(timeoutMs);
    const auto deadline = Clock::now() + timeout;

    while (true) {
        if (state.stop.load()) {
            return false;
        }

        const auto now = Clock::now();
        if (now >= deadline) {
            return false;
        }

        const auto sliceEnd = std::min(deadline, now + Milliseconds(100));
        timespec absolute{};
        if (clock_gettime(CLOCK_REALTIME, &absolute) != 0) {
            logger.raw("CLOCK_GETTIME_FAILED " +
                       std::string(std::strerror(errno)));
            state.stop.store(true);
            state.aborted.store(true);
            return false;
        }

        const long long remainingNs =
            std::chrono::duration_cast<std::chrono::nanoseconds>(
                sliceEnd - now).count();
        absolute.tv_sec += static_cast<time_t>(remainingNs / 1000000000LL);
        absolute.tv_nsec += static_cast<long>(remainingNs % 1000000000LL);
        if (absolute.tv_nsec >= 1000000000L) {
            absolute.tv_sec += 1;
            absolute.tv_nsec -= 1000000000L;
        }

        const int result = sem_timedwait(&request.event, &absolute);
        if (result == 0) {
            return true;
        }
        if (errno == EINTR) {
            continue;
        }
        if (errno == ETIMEDOUT) {
            if (Clock::now() >= deadline) {
                return false;
            }
```

```
            continue;
        }

        logger.raw("SEM_TIMEDWAIT_FAILED " +
                   std::string(std::strerror(errno)));
        state.stop.store(true);
        state.aborted.store(true);
        return false;
    }
}
```

7.4 Основной цикл философа

```
while (completed < state.config.cycles) {
    if (state.stop.load()) {
        break;
    }

    {
        std::lock_guard<std::mutex> lock(state.cs);
        state.states[id] = PhilosopherState::Thinking;
    }
    logger.event(id, "THINKING");
    std::this_thread::sleep_for(randomDuration(
        state.config.thinkMinMs, state.config.thinkMaxMs));

    if (state.stop.load()) {
        break;
    }

    Request request(id, left, right);
    if (!request.semaphoreReady) {
        logger.raw("SEM_INIT_FAILED P" + std::to_string(id));
        state.stop.store(true);
        state.aborted.store(true);
        break;
    }

    {
        std::lock_guard<std::mutex> lock(state.cs);
        if (state.stop.load()) {
            break;
        }
        state.states[id] = PhilosopherState::Waiting;
        request.enqueuedAt = Clock::now();
        state.queue.push_back(&request);
        state.maxQueue = std::max(
            state.maxQueue, static_cast<int>(state.queue.size()));
        logger.event(id, "WAITING", resources);
        dispatchLocked(state, logger);
    }

    bool granted = waitForGrant(
        state, request, state.config.timeoutMs, logger);

    if (!granted) {
        std::lock_guard<std::mutex> lock(state.cs);
        if (request.status == RequestStatus::Granted) {
```

```
            granted = true;
        } else {
            const auto position =
                std::find(state.queue.begin(), state.queue.end(), &request);
            if (position != state.queue.end()) {
                state.queue.erase(position);
            }
            request.status = RequestStatus::Cancelled;
            state.stats[id].timeouts += 1;
            state.totalTimeouts += 1;
            state.waits.push_back(WaitRecord{
                id, static_cast<double>(state.config.timeoutMs), "TIMEOUT"});
            logger.event(id, "TIMEOUT", resources);
            dispatchLocked(state, logger);
        }
    }

    if (!granted) {
        if (state.stop.load()) {
            break;
        }
        continue;
    }

    logger.event(id, "EATING");
    std::this_thread::sleep_for(randomDuration(
        state.config.eatMinMs, state.config.eatMaxMs));

    {
        std::lock_guard<std::mutex> lock(state.cs);
        state.forkOwner[left] = -1;
        state.forkOwner[right] = -1;
        state.states[id] = PhilosopherState::Thinking;
        state.stats[id].completedCycles += 1;
        logger.event(id, "RELEASED", resources);
        checkInvariantsLocked(state, logger, "release", false, false);
        dispatchLocked(state, logger);
    }

    ++completed;
}
```

7.5 Финальная проверка

```
const bool allCompleted = completedCycles == expectedCycles;
if (!allCompleted) {
    state.aborted.store(true);
}

{
    std::lock_guard<std::mutex> lock(state.cs);
    checkInvariantsLocked(state, logger, "final", true, allCompleted);
}
```

Финальная проверка требует пустую очередь, свободные вилки, состояния `FINISHED` и точное число завершённых циклов. При нарушении программа возвращает код 2 и печатает `STATUS=INVARIANT_VIOLATION`.

8 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ

8.1 Методика

Эксперименты выполнены на Ubuntu Linux, AMD Ryzen 5 5500U, 12 аппаратных потоков. Для каждой конфигурации выполнено по три запуска. Перед замерами подробный журнал отключался, чтобы операции записи не искажали время. Использовались одинаковые для всех конфигураций диапазоны имитации: обдумывание 1–2 мс, еда 20–40 мс, зерно 3716. Тайм-аут заявки — 30 000 мс, максимальная длительность — 180 000 мс.

Измерялись:

- общее время выполнения;
- пропускная способность `completed_cycles / elapsed_time`;
- среднее и максимальное время ожидания заявки;
- число тайм-аутов и обходов;
- максимальная длина очереди;
- число нарушений инвариантов и итоговый статус.

8.2 Конфигурации

Конфигурации эксперимента приведены в таблице 7. Конфигурация A является стандартной, B — индивидуальной, C — сравнением FIFO и SCAN, D — проверкой влияния числа философов.

Таблица 7 – Конфигурации эксперимента

Имя	Серия	N	Циклы	Режим	Запусков
`standard_fifo`	A	5	100	fifo	3
`individual_fifo`	B	5	240	fifo	3
`individual_scan`	C	5	240	scan	3
`load_n3_fifo`	D	3	240	fifo	3
`load_n5_fifo`	D	5	240	fifo	3
`load_n7_fifo`	D	7	240	fifo	3

8.3 Результаты

Средние результаты по трём запускам приведены в таблице 8. Значение «Макс. ожидание» — среднее по трём запускам максимальное ожидание заявки.

Таблица 8 – Средние результаты экспериментов

Конфигурация	N × циклы	Режим	Время, с	Оп/с	Ср. ожид., мс	Макс. ожид., мс	Обходы
standard_fifo	5 × 100	fifo	8.67	57.7	54.3	84.3	0
individual_fifo	5 × 240	fifo	20.76	57.8	54.6	88.4	0
individual_scan	5 × 240	scan	18.26	65.7	43.2	193.8	280.7
load_n3_fifo	3 × 240	fifo	21.55	33.4	58.1	78.3	0
load_n5_fifo	5 × 240	fifo	20.70	58.0	54.3	86.5	0
load_n7_fifo	7 × 240	fifo	20.76	80.9	54.6	91.5	0

Во всех 18 запусках: STATUS=OK, INVARIANT_VIOLATIONS=0, TIMEOUTS=0, число завершённых циклов совпало с ожидаемым. Разброс времени между запусками одной конфигурации невелик: например, для individual_fifo — 20.70–20.85 с, для individual_scan — 18.24–18.29 с.

8.4 Графики

На рисунке 5 показана пропускная способность FIFO при разном числе философов. С ростом N она увеличивается, потому что одновременно могут есть больше философов.

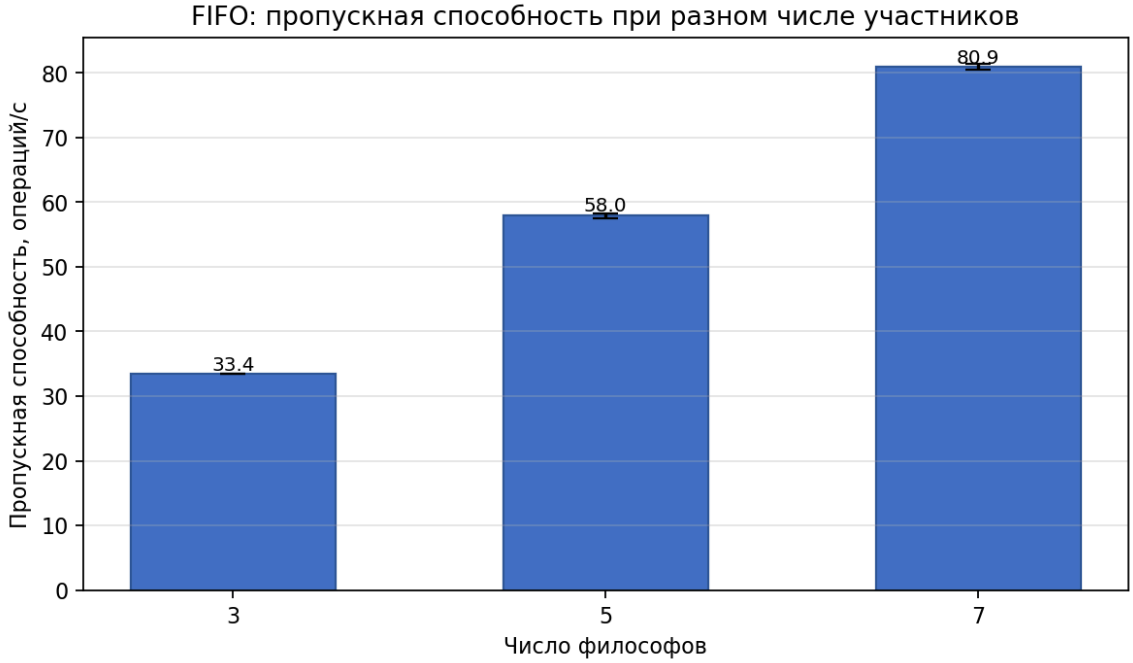


Рисунок 5 – Пропускная способность FIFO при разном числе философов

На рисунке 6 приведено распределение времени ожидания по философам для FIFO и SCAN. Видно, что FIFO даёт узкий и симметричный разброс, а SCAN — более широкий, с длинными выбросами.

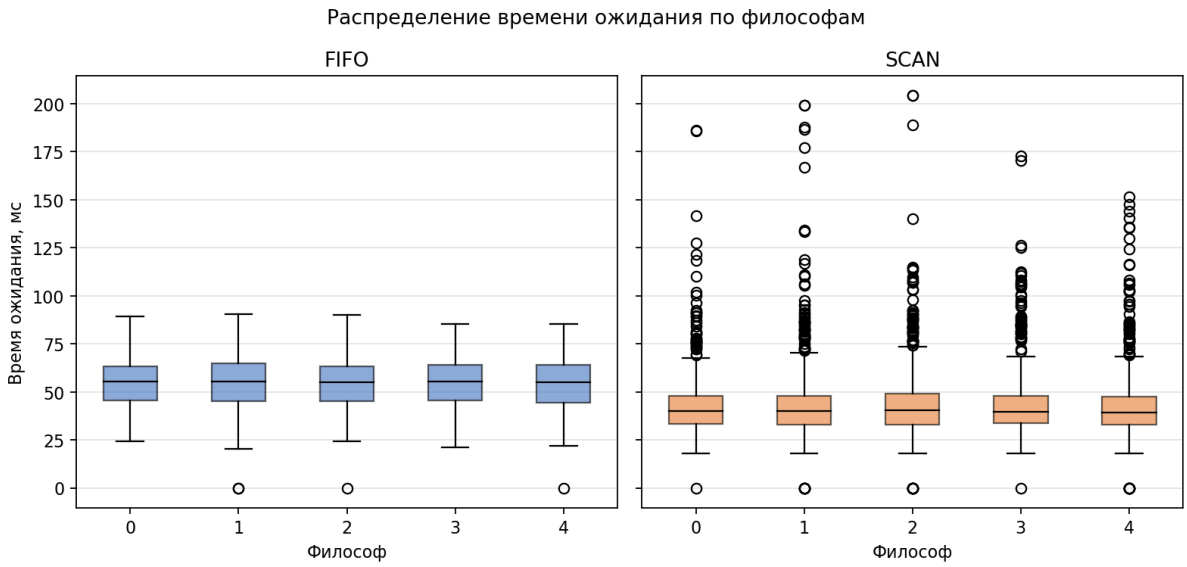


Рисунок 6 – Распределение времени ожидания по философам

На рисунках 7 и 8 показаны среднее и максимальное время ожидания. SCAN снижает среднее ожидание, но увеличивает максимальное.

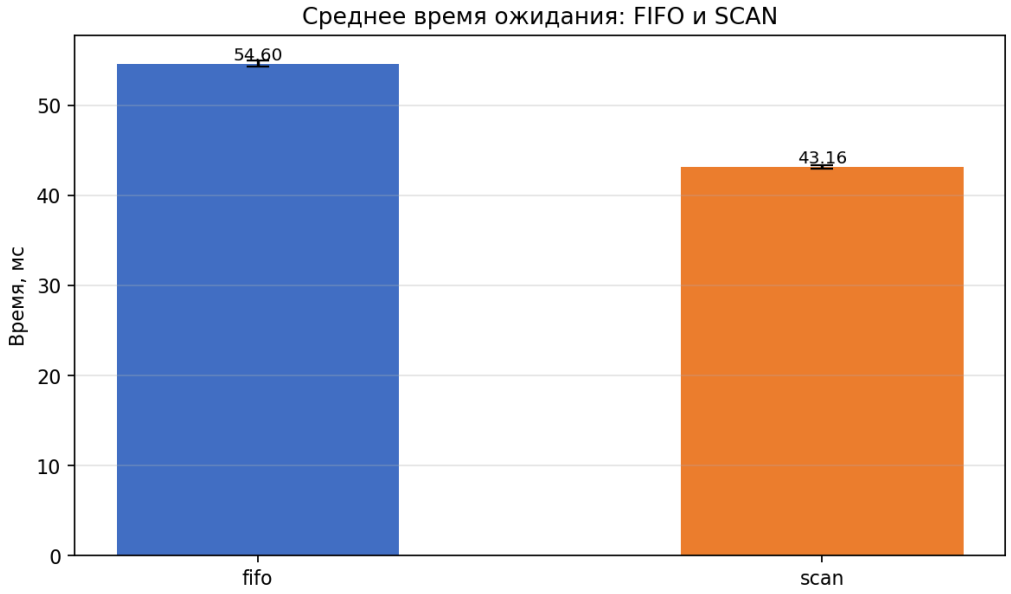


Рисунок 7 – Среднее время ожидания

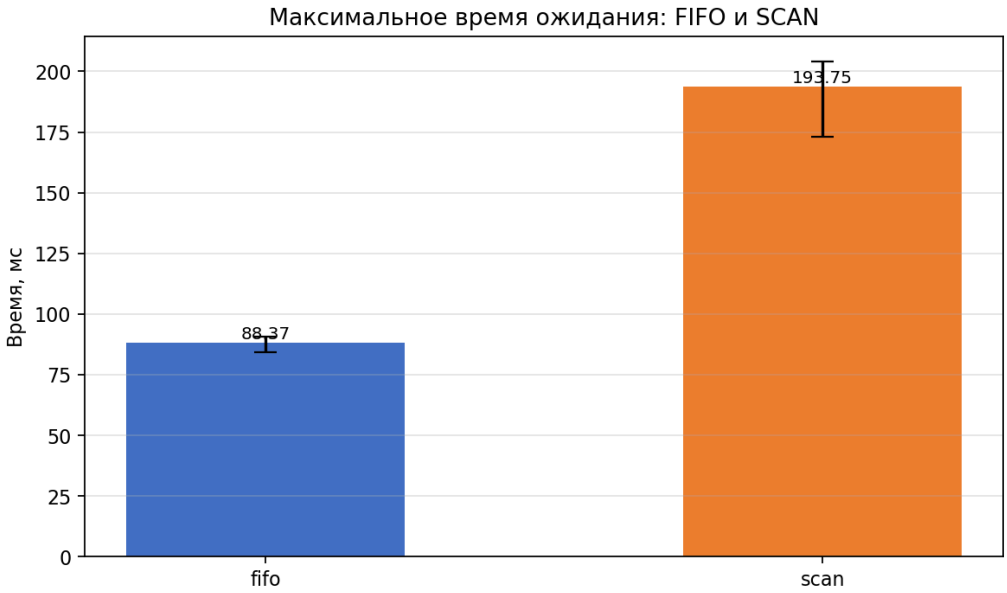


Рисунок 8 – Максимальное время ожидания

8.5 Анализ результатов

Стабильность пропускной способности. Стандартная и индивидуальная конфигурации FIFO дают практически одинаковую пропускную способность: 57.7 и 57.8 оп/с. Это ожидаемо: увеличение числа циклов в пять раз увеличивает и время выполнения примерно в пять раз, поэтому интенсивность остаётся той же. Время `standard_fifo` — 8.67 с, `individual_fifo` — 20.76 с.

FIFO и SCAN. В индивидуальной конфигурации SCAN показал 65.7 оп/с против 57.8 оп/с у FIFO, то есть примерно на 13.7 % выше. Среднее ожидание снизилось с 54.6 до 43.2 мс (примерно на 20.9 %). Однако максимальное ожидание выросло с 88.4 до 193.8 мс, а среднее число обходов составило 280.7 на запуск. Причина в том, что SCAN не ждёт заблокированную голову очереди и выдаёт вилки более поздним заявкам. Это повышает загрузку, но ухудшает худшее время ожидания и может приводить к голоданию. FIFO, наоборот, даёт предсказуемое и близкое к равномерному ожидание, но иногда простаивает, пока голова очереди не может быть обслужена.

Влияние числа философов. При $N = 3$ пропускная способность равна 33.4 оп/с, при $N = 5$ — 58.0 оп/с, при $N = 7$ — 80.9 оп/с. При $N = 3$ одновременно может есть только один философ, при $N = 5$ — два, при $N = 7$ — три. Поэтому рост пропускной способности примерно соответствует числу одновременно едящих философов. Среднее ожидание при FIFO остаётся близким к 54–58 мс, а максимальное — к 78–92 мс. Максимальная длина очереди равна $N - 1$: один философ ест, остальные ожидают.

Распределение ожидания. В таблице 9 приведены средние и максимальные времена ожидания по философам для индивидуальных конфигураций. В FIFO средние значения по философам различаются менее чем на 0.7 мс, что подтверждает отсутствие выраженного предпочтения. В SCAN различия также невелики в среднем, но максимальные значения сильно различаются и достигают 204 мс.

Таблица 9 – Время ожидания по философам

Философ	FIFO, среднее	FIFO, максимум	SCAN, среднее	SCAN, максимум
P0	54.81	89.11	42.44	186.28
P1	54.73	90.55	43.57	199.21
P2	54.70	90.21	43.99	204.23
P3	54.57	85.37	43.01	172.89
P4	54.16	85.23	42.77	151.67

Ограничения. Измерения выполнены на одной машине, число запусков равно трём, а длительности имитации малы. Результаты зависят от планировщика ОС и фоновой нагрузки. Для более строгих выводов требуется больше повторений, изоляция процессорных ядер и, возможно, отключение турбо-режима. Тем не менее качественный вывод устойчив: FIFO обеспечивает fairness, SCAN — большую пропускную способность за счёт худшего времени ожидания.

9 ЗАКЛЮЧЕНИЕ

В работе реализована модель «Обедающие философы» на C++17 для Linux с использованием POSIX-аналогов механизмов Windows API. Атомарная заявка на две вилки исключает взаимоблокировку, центральная очередь и критическая секция обеспечивают корректную синхронизацию, а персональные event-семафоры — адресное пробуждение и тайм-ауты.

Проверены инварианты, тайм-ауты, аварийное завершение и неверные аргументы. Все 18 экспериментальных запусков завершились без нарушений инвариантов и потерь циклов. Сравнение показало, что FIFO даёт справедливое и предсказуемое распределение, а SCAN повышает пропускную способность в среднем на 13.7 %, но увеличивает максимальное ожидание и число обходов. При росте числа философов пропускная способность увеличивается примерно пропорционально числу одновременно едящих философов.

Цель работы достигнута: модель реализована, корректность проверена, эксперименты воспроизводимы, результаты и графики сохранены в репозитории.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Таненбаум Э., Бос Х. Современные операционные системы. — 4-е изд. — СПб.: Питер, 2015.
2. Столлингс У. Операционные системы. — 6-е изд. — М.: Вильямс, 2013.
3. POSIX.1-2017. `sem_init`, `sem_post`, `sem_timedwait`, `clock_gettime`, `pthread_mutex_lock`, `pthread_mutex_unlock`.
4. cppreference.com. `std::thread`, `std::mutex`, `std::atomic`, `std::chrono::steady_clock`.
5. Задание на лабораторную работу №4 «Обедающие философы: справедливое распределение ресурсов». БГУИР, 2026.

ПРИЛОЖЕНИЕ А.
ИСПОЛЬЗОВАНИЕ МОДЕЛЕЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА

Раздел заполняется студентом самостоятельно.

Использование моделей искусственного интеллекта для написания, генерации, редактирования, перефразирования или оформления данного приложения запрещено условиями задания. После заполнения заменить этот блок фактическим содержанием приложения.

ПРИЛОЖЕНИЕ Б.
ЖУРНАЛ ДЕМОНСТРАЦИОННОГО ЗАПУСКА

Таблица Б.1 – Сведения о репозитории

Параметр	**Значение**
Ссылка на репозиторий	https://github.com/Dima-Vasilenko/osisp
Хеш окончательного коммита	1ee77c5970998f02ffd13fb03bfc3c9dea2dd3ed
Каталог лабораторной работы	`ossp-labs/lab04-sync`
Этап 1. Проектирование	06eca5f37c5dae9fee2e4042572b0e9366d146f1 — структура проекта, README, архитектура, диаграммы, скелет
Этап 2. Базовая реализация	99069ab1dd37dc6288471c4505b955e0ed3c3fd4 — CMake, `src/main.cpp`, CLI, состояния, центральная очередь
Этап 3. Синхронизация и тестирование	425aa59020d81cfa7e36200db59d638dba0b38b0 — `tests/run_checks.py`, CTest, инварианты, тайм-ауты, аварийные режимы
Этап 4. Эксперименты и отчёт	1ee77c5970998f02ffd13fb03bfc3c9dea2dd3ed — 18 запусков, `data/results.csv`, графики, `report.pdf`, `explained.pdf`

Ниже приведён полный журнал запуска `demo_fifo_5x3`: 5 философов по 3 цикла, режим FIFO, зерно 3716. Журнал сохранён без сокращений; файл находится в `data/raw/demo/demo_fifo_5x3.log`.

```
[000000 ms] START mode=fifo philosophers=5 cycles=3 seed=3716 student_id=28
[000000 ms] P1 START TID=800393
[000000 ms] P0 START TID=800392
[000000 ms] P1 THINKING
[000000 ms] P0 THINKING
[000000 ms] P2 START TID=800394
[000000 ms] P2 THINKING
[000000 ms] P3 START TID=800395
[000000 ms] P3 THINKING
[000000 ms] P4 START TID=800396
[000000 ms] P4 THINKING
[000001 ms] P1 WAITING RESOURCE=1,2
[000001 ms] P1 ACQUIRED RESOURCE=1,2
[000001 ms] P1 EATING
[000001 ms] P2 WAITING RESOURCE=2,3
[000001 ms] P4 WAITING RESOURCE=4,0
[000002 ms] P0 WAITING RESOURCE=0,1
[000002 ms] P3 WAITING RESOURCE=3,4
[000028 ms] P1 RELEASED RESOURCE=1,2
[000028 ms] P2 ACQUIRED RESOURCE=2,3
[000028 ms] P4 ACQUIRED RESOURCE=4,0
[000028 ms] P1 THINKING
[000028 ms] P4 EATING
[000028 ms] P2 EATING
[000029 ms] P1 WAITING RESOURCE=1,2
[000048 ms] P4 RELEASED RESOURCE=4,0
[000048 ms] P0 ACQUIRED RESOURCE=0,1
[000048 ms] P4 THINKING
[000048 ms] P0 EATING
```

```
[000049 ms] P4 WAITING RESOURCE=4,0
[000067 ms] P2 RELEASED RESOURCE=2,3
[000067 ms] P3 ACQUIRED RESOURCE=3,4
[000067 ms] P2 THINKING
[000067 ms] P3 EATING
[000068 ms] P2 WAITING RESOURCE=2,3
[000070 ms] P0 RELEASED RESOURCE=0,1
[000070 ms] P1 ACQUIRED RESOURCE=1,2
[000070 ms] P0 THINKING
[000070 ms] P1 EATING
[000072 ms] P0 WAITING RESOURCE=0,1
[000104 ms] P1 RELEASED RESOURCE=1,2
[000104 ms] P1 THINKING
[000105 ms] P3 RELEASED RESOURCE=3,4
[000105 ms] P4 ACQUIRED RESOURCE=4,0
[000105 ms] P2 ACQUIRED RESOURCE=2,3
[000105 ms] P3 THINKING
[000105 ms] P4 EATING
[000105 ms] P2 EATING
[000106 ms] P3 WAITING RESOURCE=3,4
[000107 ms] P1 WAITING RESOURCE=1,2
[000126 ms] P2 RELEASED RESOURCE=2,3
[000126 ms] P2 THINKING
[000128 ms] P2 WAITING RESOURCE=2,3
[000132 ms] P4 RELEASED RESOURCE=4,0
[000132 ms] P0 ACQUIRED RESOURCE=0,1
[000132 ms] P3 ACQUIRED RESOURCE=3,4
[000132 ms] P4 THINKING
[000132 ms] P0 EATING
[000133 ms] P3 EATING
[000135 ms] P4 WAITING RESOURCE=4,0
[000163 ms] P0 RELEASED RESOURCE=0,1
[000163 ms] P1 ACQUIRED RESOURCE=1,2
[000163 ms] P0 THINKING
[000163 ms] P1 EATING
[000164 ms] P0 WAITING RESOURCE=0,1
[000171 ms] P3 RELEASED RESOURCE=3,4
[000171 ms] P3 THINKING
[000173 ms] P3 WAITING RESOURCE=3,4
[000183 ms] P1 RELEASED RESOURCE=1,2
[000183 ms] P2 ACQUIRED RESOURCE=2,3
[000183 ms] P4 ACQUIRED RESOURCE=4,0
[000183 ms] P1 FINISHED
[000183 ms] P4 EATING
[000183 ms] P2 EATING
[000220 ms] P2 RELEASED RESOURCE=2,3
[000220 ms] P2 FINISHED
[000223 ms] P4 RELEASED RESOURCE=4,0
[000223 ms] P0 ACQUIRED RESOURCE=0,1
[000223 ms] P3 ACQUIRED RESOURCE=3,4
[000223 ms] P4 FINISHED
[000223 ms] P0 EATING
[000223 ms] P3 EATING
[000251 ms] P3 RELEASED RESOURCE=3,4
[000251 ms] P3 FINISHED
[000254 ms] P0 RELEASED RESOURCE=0,1
[000254 ms] P0 FINISHED
```